Structure formation through a purely repulsive potential: a Monte Carlo study

Jakob Eisenhauer

September 25, 2026

Abstract

This report studies, by Metropolis Monte Carlo simulation, a two dimensional system of particles interacting through a purely repulsive hard-core-corona pair potential. The model was introduced by Malescio and Pellicane (2003) to show that stripe patterns can arise from a purely repulsive interaction with two length scales. Final configurations are obtained at fixed temperature $T^* = 0.1$ for five densities, and a temperature sweep at fixed density $\rho^* = 0.291$ is used to locate a transition into a stripe ordered phase. The results are compared, mainly on a qualitative level, to those reported in the original Nature Materials publication.

1 Introduction

Malescio and Pellicane [1] showed that a two dimensional system of particles interacting only through a purely repulsive potential with two characteristic length scales, a hard core of diameter σ_0 surrounded by a soft, penetrable corona of diameter σ_1, forms a sequence of distinct equilibrium structures depending on density, and undergoes a transition into a striped, orientationally ordered phase when the temperature is lowered at fixed density. In this project the same model is studied with the Metropolis Monte Carlo method, both at fixed low temperature for a range of densities and along a gradual cooling schedule at fixed density.

Section 2 describes the model. Section 3 summarises the Metropolis Monte Carlo method and the mechanism behind it. Section 4 describes the simulation set up. Section 5 presents the results together with their discussion. Section 6 concludes. The simulation code is reproduced in full in Appendix A; the same code, together with the raw output data, is available on GitHub at `https://github.com/J-Eisenhauer/MCMC_project`.

2 Model

N particles are confined to a two dimensional square box of side length L, subject to periodic boundary conditions, so that the simulated system approximates a much larger system despite the modest number of particles that can be simulated explicitly. All simulations in this report use $N = 250$ particles.

Particles interact pairwise through the radially symmetric potential

$$\phi(r) = \begin{cases} \infty, & r < \sigma_0 \\ \varepsilon, & \sigma_0 \le r < \sigma_1 \\ 0, & r \ge \sigma_1 \end{cases} \tag{1}$$

where r is the interparticle separation, σ_0 is the hard-core diameter, σ_1 is the diameter of the soft

corona surrounding it, and ε is the finite energy penalty for two coronas overlapping. Following Malescio and Pellicane [1], $\sigma_1/\sigma_0 = 2.5$ is used throughout.

Reduced units are used throughout, with σ_0 as the unit of length and ε as the unit of energy: the reduced temperature is $T^* = k_B T/\varepsilon$ and the reduced density is $\rho^* = N\sigma_0^2/L^2$. For a given N and ρ^*, the box length follows from $L = \sigma_0\sqrt{N/\rho^*}$. The simulation is performed in the canonical (N, V, T) ensemble: N, $V = L^2$ and T are all held fixed within a given run, and only particle positions are updated.

Distances are evaluated using the minimum image convention, so that each particle interacts with the nearest periodic copy of every other particle.

3 The Metropolis Monte Carlo method

Metropolis Monte Carlo [2] is a Markov chain Monte Carlo technique for drawing samples from a target probability distribution when that distribution's normalisation is unknown or too costly to evaluate directly, which is the situation for the Boltzmann distribution

$$P_B(\mathbf{r}^N) = \frac{e^{-\beta U(\mathbf{r}^N)}}{\int e^{-\beta U(\mathbf{r}^N)}\, d\mathbf{r}^N}, \qquad \beta = \frac{1}{k_B T}, \tag{2}$$

of an N particle system at fixed volume and temperature: the partition function in the denominator can only be evaluated for the smallest systems. Instead of sampling P_B directly, Metropolis Monte Carlo constructs a Markov chain of configurations whose stationary distribution is P_B, so that time averages over a long chain converge to ensemble averages.

This is achieved by proposing a trial move from the current configuration m to a new configuration n with probability $\alpha(m \to n)$, and then accepting it with probability $\mathrm{acc}(m \to n)$. Writing $\pi_m \propto e^{-\beta U_m}$ for the Boltzmann weight of configuration m, detailed balance requires $\pi_m\, \alpha(m \to n)\, \mathrm{acc}(m \to n) = \pi_n\, \alpha(n \to m)\, \mathrm{acc}(n \to m)$. For symmetric proposals, $\alpha(m \to n) = \alpha(n \to m)$, this is satisfied by the Metropolis acceptance criterion

$$\mathrm{acc}(m \to n) = \min\left(1,\ \frac{\pi_n}{\pi_m}\right) = \min\left(1,\ e^{-\beta \Delta U}\right), \qquad \Delta U = U_n - U_m, \tag{3}$$

so that a move which lowers the energy is always accepted, while an energy increasing move is accepted with probability $e^{-\beta \Delta U}$. Repeating this over many steps generates a chain of configurations that is asymptotically Boltzmann distributed, from which thermodynamic averages can be estimated once the chain has equilibrated.

In this system, one Monte Carlo step (one attempted configuration, the unit in which all run lengths in this report are quoted) consists of selecting a particle i at random, proposing a trial displacement $x_i \to x_i + \delta\, \xi_x$ and $y_i \to y_i + \delta\, \xi_y$, where ξ_x, ξ_y are independent random numbers uniform on $[-1, 1]$ and δ is a maximum step size, wrapping the new coordinates back into $[0, L)$, computing the resulting energy change ΔU of particle i with all other particles before and after the move, and finally accepting the move with probability $\min(1, e^{-\beta \Delta U})$ or otherwise restoring the old coordinates. Because the potential includes a hard core, a proposed move that would place two particles closer than σ_0 is always rejected without evaluating the Metropolis criterion numerically, since it corresponds to an infinite energy increase.

The maximum step size δ controls the acceptance ratio: too small a step gives high acceptance but slow exploration of configuration space, too large a step leads to most moves being rejected. Here δ is tuned separately for each thermodynamic state, aiming for an acceptance ratio of approximately 30% (Section 5.2).

4 Simulation set-up and implementation

All simulations were implemented in Python. The code is organised into three scripts. `mc_fixed_temperature.py` contains the core Monte Carlo routines and produces the fixed temperature density scan discussed in Section 5.3. `mc_temperature_drop_parallel.py` imports those routines and drives the ten parallel annealing chains discussed in Section 5.4. `plot_results.py` turns the saved output of both scripts into the figures shown in this report.

A pure Python implementation of the Metropolis algorithm described above is far too slow to reach the number of attempted moves needed for a meaningful production run. Two optimisations are used throughout. The performance critical routines are compiled to machine code with Numba's `@njit` decorator [3] ahead of the main run. In addition, rather than recomputing the total interaction energy from scratch after every attempted move, which scales as $O(N^2)$, the running total energy is tracked, and only the interaction energy of the moved particle with the remaining $N-1$ particles is recomputed, before and after the proposed displacement. This reduces the cost of a single attempted move to $O(N)$.

Every state is simulated in three stages: a tuning run for the maximum displacement (Section 5.2), an equilibration run of 5×10^6 attempted configurations, and a production run. For the fixed temperature density scan, 1×10^8 production configurations are used for each of the five densities. The temperature sweep at $\rho^* = 0.291$ is considerably more demanding, since it requires an equilibration and production run at every temperature step, repeated for each of the ten chains. The number of production configurations per temperature is not uniform: the windows $0.225 \leq T^* \leq 0.25$, $0.20 \leq T^* < 0.225$, $0.17 \leq T^* < 0.20$ and $0.15 \leq T^* < 0.17$ are given 5×10^7, 1.5×10^8, 3×10^8 and 1.5×10^8 production configurations respectively, so that the largest effort is concentrated where the transition is expected and where the fluctuations, and hence the statistical noise in the heat capacity, are largest. Every state studied therefore receives at least 5×10^6 equilibration and 5×10^7 production configurations. The spacing of the simulated temperatures is likewise refined near the expected transition: $T^* = 0.25, 0.24, \ldots, 0.21$ in steps of 0.01, then $T^* = 0.205, 0.203, \ldots, 0.175$ in steps of 0.002, and finally $T^* = 0.165$, 0.155 and 0.150, giving 24 temperatures in total. The ten chains, which differ only in their random seed and otherwise follow the same schedule, are run as separate processes with Python's `multiprocessing.Pool`.

5 Results and discussion

5.1 Approach to equilibrium and energy conservation

Figure 1 shows the energy per particle during the equilibration run at $T^* = 0.1$, for each of the five densities studied. In every case the energy falls sharply at first, as the random initial configuration relaxes into a lower energy arrangement, and then converges more slowly towards a stable value. The final energy per particle increases with density, since a denser packing forces more corona overlaps.

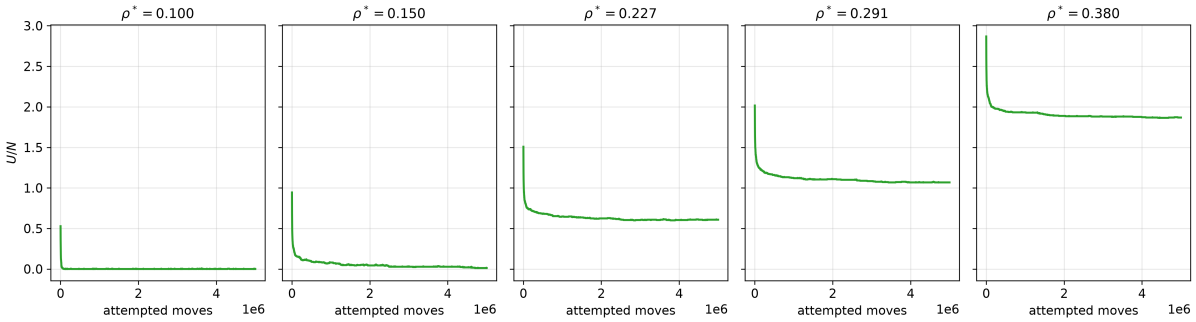


Figure 1: Energy per particle, U/N, versus the number of attempted configurations, sampled every 1000 moves, during the equilibration run at $T^* = 0.1$ for each of the five densities.

Energy conservation was checked as an internal consistency test of the code. At the end of each production run, the incrementally tracked total energy was compared with the total energy recomputed independently through the full $O(N^2)$ sum over all pairs. Because every accepted move changes U by an exact integer multiple of $\varepsilon = 1$, the two values are expected to agree exactly, and any nonzero difference would indicate an error in the incremental update. Across all 240 production runs of the temperature sweep and all five production runs of the density scan, the tracked and recomputed energies agreed exactly, i.e. the energy drift $U_{\text{tracked}} - U_{\text{recomputed}}$ was zero in every case.

5.2 Acceptance ratios

The maximum displacement δ is tuned iteratively towards an acceptance ratio between 25% and 35%. In the density scan, the tuning run follows the equilibration run and precedes the production run. In the temperature sweep, where each state starts from the final configuration of the previous, warmer temperature, the tuning run comes first, and the tuned displacement is used for both the equilibration and the production run at that temperature. The tuning run has its own short equilibration: the system is first allowed to relax for 1×10^5 discarded moves at the current displacement. The displacement is then tested in blocks of 10^4 attempted moves and rescaled by a factor of 0.8 if the measured acceptance falls below 25%, or by 1.2 if it exceeds 35%. Tuning stops at the first block that falls inside the target range once at least three blocks have been run, or after eight blocks at the latest. The resulting displacement is then kept fixed for the rest of that state.

Across all 240 production runs of the temperature sweep, the acceptance ratio ranged from 24.9% to 34.4%, with a mean of 29.1%. For the five production runs of the fixed-temperature density scan, the acceptance ratio ranged from 22.2% to 33.7%, with a mean of 28.3%. The mean values lie in the middle of the target range; the two production runs below 25%, one run of the temperature sweep and $\rho^* = 0.380$ in the density scan, reflect that the acceptance measured over a whole production run can differ from that of the short tuning blocks of 10^4 moves, since the configuration keeps evolving after the tuning has finished.

5.3 Final structures and radial distribution functions at $T^* = 0.1$

Figure 2 shows, for each of the five densities studied, the final configuration reached after the equilibration and production run at $T^* = 0.1$, together with the corresponding radial distribution function. Each panel is an independent simulation at fixed density and temperature, so the five columns should be read as a comparison between separate final states rather than as one system evolving with density.

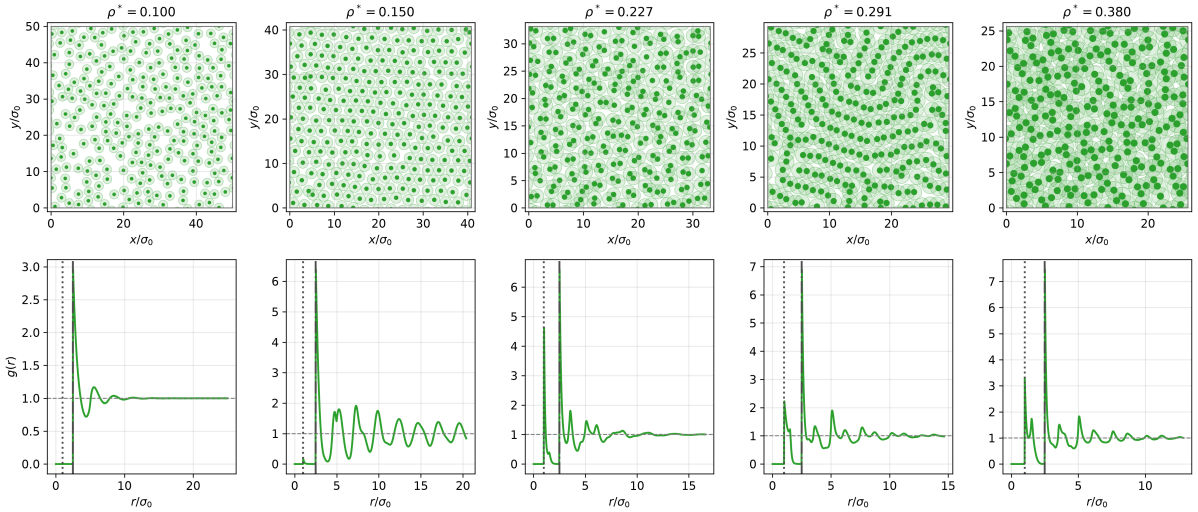


Figure 2: Final configuration (top row) and radial distribution function $g(r)$ (bottom row) at $T^* = 0.1$, for each of the five densities studied. The dotted and dash-dotted vertical lines in the bottom row mark σ_0 and σ_1, respectively.

At $T^* = 0.1$, a move that creates one additional corona overlap is energetically unfavourable compared to the current configuration and is accepted only with probability $e^{-1/0.1} = e^{-10} \approx 5 \times 10^{-5}$. Almost every overlap seen in the snapshots is therefore one that the available space forces the system to make, or one that was frozen in while the random initial configuration relaxed (Section 5.5). Since each overlapping pair contributes ε to the energy, $2U/N$ is the average number of overlap partners per particle, which can be read off Fig. 1.

At the lowest density, $\rho^* = 0.100$, the particles are well separated and $g(r)$ shows essentially one feature, a single peak located at $r = \sigma_1$. At this density there is enough room for particles to avoid any corona overlap entirely, and the closest distance that carries no energy penalty, $r = \sigma_1$, is the one that is statistically favoured: a particle that sits close to σ_1 from one neighbour leaves more free space, and hence larger distances, to all other particles, which keeps the probability of an overlap with any of them low. No peak appears at $r = \sigma_0$, since paying the corresponding energy penalty is not necessary here.

At $\rho^* = 0.150$, the particles form a regular, lattice-like arrangement. At this density the available space is only slightly larger than what the particles need to keep a distance of σ_1 from each other, so the particles almost have to overlap. The only way to avoid overlaps is a uniform, ordered configuration in which every particle keeps a distance of about σ_1 to all of its neighbours. This order shows up in $g(r)$ as a dominant peak at σ_1 followed by a long, periodic sequence of sharp peaks that persists across the box, while the peak at σ_0 is almost absent.

At $\rho^* = 0.227$, corona overlaps can no longer be avoided, and the particles form pairs with their hard cores in contact, together with a few short chains. As argued by Malescio and Pellicane [1], if an overlap is unavoidable it is cheapest to bring the hard cores of the two particles into direct contact: the pair then pays for a single overlap, while occupying little space, so that all other particles can be kept at a distance of at least σ_1. Consistently, $U/N \approx 0.6$, i.e. each particle has on average about one overlap partner. $g(r)$ now shows two distinct peaks, one at σ_0 from the partner within a pair, where the penalty is paid, and one at σ_1 from the neighbouring pairs, where it is avoided.

At $\rho^* = 0.291$, the limited space makes two overlaps per particle necessary, and two overlaps with different particles naturally connect the particles into chains. These chains form the winding

filaments of the stripe phase. To use the penalty free space as efficiently as possible, neighbouring chains lie almost exactly σ_1 apart, which produces the strong peak at σ_1. The energy confirms this picture: $U/N \approx 1.07$, i.e. each particle overlaps with about two others, its two neighbours in the chain. Within a chain the particles are not always in core contact, so $g(r)$ shows a peak at σ_0 with a shoulder up to about $1.6\,\sigma_0$, and no neighbours at all between about $1.7\,\sigma_0$ and σ_1.

At the highest density, $\rho^* = 0.380$, space is so limited that particles overlap with more than two others, and the configuration consists of small, compact islands, many of them of three particles, with the hard cores in contact inside each island. Grouping the unavoidable overlaps into tightly packed islands frees space between them, and neighbouring islands keep a distance of about σ_1 where possible, so the peak at σ_1 remains the tallest, while the peak at σ_0 is more pronounced than at $\rho^* = 0.291$. With $U/N \approx 1.87$, i.e. almost four overlap partners per particle, not all islands can be kept σ_1 apart, so overlaps between neighbouring islands also occur.

Across the density scan, the structure is therefore controlled by the competition between the two length scales of the potential: the particles arrange themselves on the scale σ_1 wherever space allows, and the overlaps that the density forces on them are concentrated in core contacts on the scale σ_0, in pairs, chains and islands of increasing connectivity.

5.4 Temperature dependence of the heat capacity at $\rho^* = 0.291$

Starting at $T^* = 0.25$, the temperature is lowered in the non-uniform steps described in Section 4, down to $T^* = 0.15$, using ten independent chains that differ only in their random seed. Each temperature starts from the final configuration of the previous one. Figure 3 shows the configuration of one of the chains at five points along this schedule. At $T^* = 0.25$ the configuration already consists of curved, randomly oriented filaments. The reorganisation is not a gradual process: for this realisation, above $T^* \approx 0.20$ the structure hardly changes, and most of the reorganisation takes place within the narrow interval between $T^* \approx 0.20$, where the stripes begin to align in parts of the box, and $T^* \approx 0.19$, where almost the whole box consists of straight, parallel stripes. The few remaining defects disappear on further cooling, and below $T^* \approx 0.18$ the system is arranged in long, straight, parallel stripes spanning the box. Following Malescio and Pellicane [1], this is a transition from a disordered to an orientationally ordered state: the filaments exist on both sides of the transition, and what becomes ordered is their direction, from randomly curved to straight and parallel.

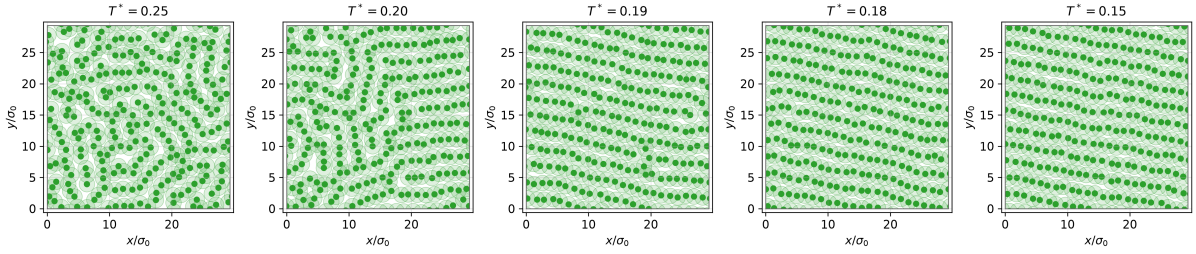


Figure 3: Configuration of one representative chain (realisation 1) at five points along the cooling schedule, at $\rho^* = 0.291$.

The heat capacity is obtained from the energy fluctuations in the production run through $C_V = (\langle U^2 \rangle - \langle U \rangle^2)/(k_B T^2)$, using one energy sample every 1000 attempted moves. Figure 4 shows the resulting heat capacity per particle for each of the ten chains individually, together with the mean and the standard error of the mean, computed by treating the ten chains as independent measurements at each temperature. The individual chains scatter considerably close to the transition. At $T^* = 0.191$, for example, the measured heat capacity per particle ranges from

about 6.9 to 28.3 across the ten chains. Despite this scatter, the mean curve shows a single, sharp peak: the largest mean value, 17.5 ± 2.0, is found at $T^* = 0.191$, and the neighbouring point at $T^* = 0.189$ is similarly high, at 14.2 ± 2.5, so the peak of the mean curve is best described as lying between these two temperatures. Below the peak the heat capacity drops steeply, and by $T^* \approx 0.183$, where the stripes in Fig. 3 have become straight and aligned, it has fallen back below 3.

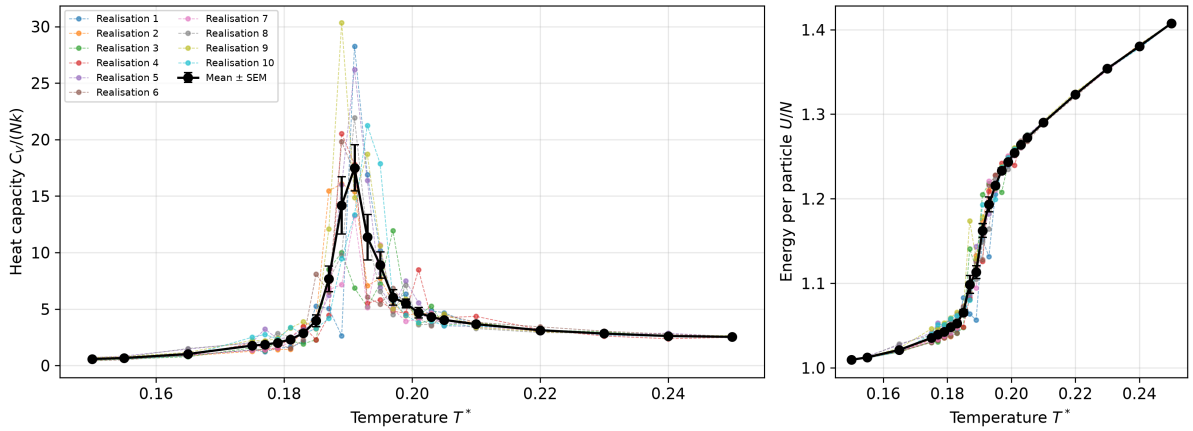


Figure 4: Heat capacity per particle $C_V/(Nk_B) = \left(\langle U^2\rangle - \langle U\rangle^2\right)/(NT^{*2})$ (left) and energy per particle U/N (right), with U the potential energy in units of ε, for each of the ten independent chains (realisations 1–10) at $\rho^* = 0.291$, together with the mean and the standard error of the mean (SEM) across chains at each temperature.

The energy per particle (Fig. 4, right) shows the same transition from a different angle. It decreases smoothly and almost linearly with temperature above $T^* \approx 0.2$ and below $T^* \approx 0.18$, but the mean drops abruptly in between, from $U/N \approx 1.25$ to $U/N \approx 1.05$, i.e. exactly in the temperature range of the heat capacity peak. The individual chains do not undergo this drop at exactly the same temperature: some, such as realisation 1, already reach the low-energy stripe state at $T^* = 0.193$, while others only settle there at $T^* \approx 0.185$. This spread in the transition temperature between chains is the same effect that produces the scatter in the heat capacity close to the transition, where the individual peaks vary in position and height. A plausible explanation for this scatter is that, close to the transition, a chain can become effectively trapped in one local arrangement, for example a particular orientation or number of stripes, for a duration comparable to the length of the production run at that temperature. Such slow relaxation is expected near a phase transition, and the sampled average is then biased towards that particular arrangement rather than the true thermal average. A second contribution comes from the fact that only the production run enters the energy statistics. If a chain completes its reorganisation into the stripe state during the equilibration run, the associated large change in energy is never recorded. If, however, it remains in a metastable arrangement throughout the equilibration run and only reorganises during the production run, the drift of the energy from one branch to the other enters $\langle U^2\rangle - \langle U\rangle^2$ and strongly increases the measured heat capacity. Consistently, all production runs with $C_V/(Nk_B) > 18$ have a mean energy between the two branches, $U/N \approx 1.11$ to 1.19, compared to $U/N \approx 1.20$ to 1.24 just above and $U/N \approx 1.04$ to 1.08 just below the transition. The length of the equilibration run relative to the relaxation time of the system therefore influences the recorded energy variance close to the transition.

5.5 Comparison with Malescio and Pellicane (2003)

The results agree qualitatively, in most respects, with those reported by Malescio and Pellicane [1].

Structures at $T^* = 0.1$. The configurations in Fig. 2 follow the same sequence as Figure 2 of Ref. [1]: a disordered arrangement at $\rho^* = 0.100$, a triangular lattice with lattice constant σ_1 and few defects at $\rho^* = 0.150$, mainly dimers and a few short chains at $\rho^* = 0.227$, stripe domains at $\rho^* = 0.291$, and loose aggregates of three or more particles at $\rho^* = 0.380$. Malescio and Pellicane characterise the structure through the structure factor $S(k)$ rather than $g(r)$, so the curves cannot be compared directly, but the trends agree. They find that at low density $S(k)$ is dominated by its first peak at $k \approx 2\pi/\sigma_1$, i.e. σ_1 is the relevant length scale, that an additional, shorter length scale emerges at intermediate densities, and that σ_0 only becomes the dominant length scale at densities above those studied here. Correspondingly, in the present $g(r)$ the peak at σ_1 remains the tallest at all five densities, while core contacts at σ_0 appear from $\rho^* = 0.227$ onwards.

Malescio and Pellicane note that their Figure 2 configurations were obtained by cooling relatively quickly and may therefore contain frozen-in disorder. The same applies to the configurations in Fig. 2, which were started directly at $T^* = 0.1$ from a random configuration. This is visible at $\rho^* = 0.291$: the configuration obtained directly at $T^* = 0.1$ consists of winding stripe domains, whereas the slowly cooled chains of Section 5.4 form nearly defect-free, straight, parallel stripes spanning the box. The energy makes this quantitative: the configuration obtained directly at $T^* = 0.1$ has $U/N \approx 1.07$ (Fig. 1), whereas the slowly cooled chains reach $U/N \approx 1.01$ already at $T^* = 0.15$ (Fig. 4). Since the mean energy can only decrease further on cooling, the equilibrium energy at $T^* = 0.1$ lies at or below 1.01, so the directly obtained configuration at $\rho^* = 0.291$ is metastable rather than in equilibrium.

Transition at $\rho^* = 0.291$. The temperature-driven reorganisation into stripes is also found here. In Figure 3 of Ref. [1], winding filaments at $T = 0.19$ turn into nearly straight stripes spanning the system at $T = 0.18$; in Fig. 3 the same reorganisation is already largely complete at $T^* \approx 0.19$, consistent with the slightly higher temperature of the heat capacity peak found here. The mean heat capacity peaks between $T^* = 0.189$ and $T^* = 0.191$, at or slightly above the upper end of the range $0.18 < T_S < 0.19$ reported by Malescio and Pellicane for the transition temperature T_S into the stripe phase. Given the width of the peak and the scatter between chains this is reasonable agreement, and the small offset is plausibly a finite-size effect: with only $N = 250$ particles the peak of the heat capacity is not expected to sit exactly at the transition temperature of the macroscopic system (Section 5.6). Away from the transition, the heat capacity also agrees quantitatively with Figure 4 of Ref. [1], with values of approximately 2.5 at $T^* = 0.25$ in both studies and a steep drop below the transition in both. The height of the peak differs: the mean value found here, 17.5 ± 2.0, is clearly larger than the value of approximately 5 at $T = 0.19$ in Ref. [1]. A likely contribution is that some chains only reorganise into the stripe state during the production run rather than during the equilibration run, so that the resulting energy drift is recorded and increases the measured heat capacity (Section 5.4). The energy per particle also has the same form as in the inset of Figure 4 of Ref. [1]: two smooth branches above and below the transition, joined by an abrupt decrease at T_S.

5.6 Limitations and sources of error

Several limitations should be kept in mind when interpreting the results above. The first is finite sampling, of which the scatter between chains discussed in Section 5.4 is one instance. It also affects the error bars in Fig. 4: the standard error treats all ten chains as equally informative independent samples, although a chain that is trapped in a local arrangement contributes less

independent information than one that decorrelates freely, so the true uncertainty near the peak is likely larger than shown. In addition, successive energy samples within a run are correlated, which is not a problem for the averages but means that the effective number of independent samples is smaller than the number recorded. Close to the transition, the measured heat capacity also depends on whether a chain reorganises during the equilibration or the production run (Section 5.4), so the peak height is sensitive to the length of the equilibration run. The configurations at $T^* = 0.1$ were started directly from a random configuration and may contain frozen-in disorder, as shown explicitly for $\rho^* = 0.291$ in Section 5.5; at $\rho^* = 0.150$ the energy is also still decreasing slightly at the end of the equilibration run (Fig. 1). The finite temperature step used in the annealing schedule is a discrete approximation to continuous, quasi-static cooling; if the steps are too coarse relative to the local relaxation time of the system, the system lags behind the imposed temperature.

The finite size of the system, $N = 250$, is expected to shift the transition temperature somewhat and to round off and broaden the peak in the heat capacity relative to the sharp feature expected in the thermodynamic limit. The stripe pattern observed at high density and low temperature must also be geometrically compatible with the size and shape of the simulation box under periodic boundary conditions. A box whose side length does not fit a whole number of stripes can distort the pattern, or favour or disfavour it, which is a further finite-size effect tied to the geometry of the simulation.

Finally, at the simulated density the true equilibrium state of the system may lie within a two-phase coexistence region of the full phase diagram. In a simulation at fixed average density, such phase separation can be suppressed by the finite size of the system, since a genuinely macroscopic system would be free to separate into coexisting domains of different density, while the system studied here is constrained to a comparatively small periodic box. Malescio and Pellicane [1] address this concern directly by additionally performing simulations at constant pressure, in which the volume is allowed to fluctuate and the system is therefore free to relax towards its true equilibrium density, and they report agreement with their constant volume results. No such constant pressure simulations were performed here, so it cannot be ruled out that the fixed density results are affected by proximity to a two-phase coexistence region.

6 Conclusion

Using Metropolis Monte Carlo simulations, this project reproduced the qualitative phenomenology of the hard-core-corona model of Malescio and Pellicane [1]: a density dependent sequence of structures at fixed low temperature, and a temperature driven reorganisation into stripes at fixed density, with the mean heat capacity across ten independent chains peaking between $T^* = 0.189$ and $T^* = 0.191$, at or slightly above the upper end of the transition temperature range of 0.18 to 0.19 reported in the original study. The considerable scatter between individual chains close to the transition shows that a single run of this length is not sufficient to pin down the transition temperature precisely, and is attributed to finite-size and finite-sampling effects.

AI clarification

AI assistance was used in this project in the following three ways.

Code. In `mc_fixed_temperature.py` and `mc_temperature_drop_parallel.py`, AI was used to detect and fix bugs, to help implement the parallel execution of the ten chains, and for advice on writing the code in a clear, Pythonic style. The plotting script `plot_results.py` relied more heavily on AI assistance, in particular for the true-to-scale particle rendering with periodic images. I checked and tested all code and am responsible for it.

Writing. AI was used for language editing and rephrasing at the level of individual sentences, and for the typesetting of the report in LaTeX.

Review. Drafts of the report were reviewed by AI, and its feedback prompted some revisions. I decided which suggestions to adopt and checked everything I adopted.

The implementation of the model, the simulation design, the analysis and the interpretation of the results are my own work. The models used were Claude Sonnet 5 and Claude Opus 5.5.

References

[1] Gianpietro Malescio and Giuseppe Pellicane. Stripe phases from isotropic repulsive interactions. *Nature Materials*, 2(2):97–100, 2003. doi: 10.1038/nmat820.

[2] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114.

[3] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, pages 7:1–7:6, New York, 2015. ACM. doi: 10.1145/2833157.2833162.

A Full source listing

For completeness, the three scripts are reproduced here in full: the core Monte Carlo routines and fixed-temperature density scan (`mc_fixed_temperature.py`), the parallel temperature-drop driver (`mc_temperature_drop_parallel.py`), and the plotting script that produces every figure in this report from their saved output (`plot_results.py`).

mc_fixed_temperature.py

```
"""Fixed-temperature part of the stripe-phase Monte Carlo project.

Runs N particles at T* = 0.1 for the five densities and saves:
  - the equilibration energy trace,
  - the final configuration,
  - the production-averaged radial distribution function g(r),
  - a small text summary for numerical checks and acceptance ratios.

All plotting for this project lives in plot_results.py, so this script only
simulates and writes data to disk.
"""


from pathlib import Path
import time
import numpy as np
from numba import njit


# ----------------------------- settings -----------------------------
N = 250 # number of particles
DENSITIES = (0.100, 0.150, 0.227, 0.291, 0.380) # rho* values
TEMPERATURE = 0.1 # reduced temperature kT/epsilon
SIGMA_0 = 1.0 # hard-core distance
SIGMA_1 = 2.5 # shoulder distance
MAX_DISPLACEMENT = 0.5 # trial displacement in each direction
TUNING_BURN_IN_STEPS = 100_000 # discarded before acceptance is assessed
TUNING_STEPS = 10_000 # extra pre-equilibration moves per test
MIN_TUNING_ROUNDS = 3 # min number of tuning blocks for max_displacement
MAX_TUNING_ROUNDS = 8 # max number of tuning blocks for max_displacement
MIN_ACCEPTANCE = 0.25 # desired lower acceptance-rate limit.
MAX_ACCEPTANCE = 0.35 # desired upper acceptance-rate limit.

EQUILIBRATION_STEPS = 5_000_000 # number of moves before production (for fixed T* runs)
PRODUCTION_STEPS = 100_000_000 # number of moves for production (for fixed T* runs)

ENERGY_EVERY = 1_000 # Energy is recorded every ENERGY_EVERY attempted moves
RDF_EVERY = 1_000 # RDF = Radial Distribution Function

RDF_BINS = 1_000 # Fine resolution between r=0 and L/2.
OUTPUT_DIR = Path("fixed_temperature_results")
BASE_SEED = 2026



# ------------------------- Monte Carlo functions -------------------------
@njit
def minimum_image(delta, box_length):
    """
    Return the shortest signed separation in a periodic square box.

    np.rint rounds to the nearest integer. Subtracting this integer number of
    box lengths maps a displacement into approximately [-L/2, L/2].
```

```
    """
    return delta - box_length * np.rint(delta / box_length)


@njit
def particle_energy(x, y, particle, box_length):
    """
    Calculate how one particle interacts with every other particle.

    In reduced units, each soft overlap contributes epsilon = 1.
    Returning -1 is a fast signal that a forbidden hard-core overlap exists.

    """
    energy = 0
    sigma0_sq = SIGMA_0**2
    sigma1_sq = SIGMA_1**2

    for other in range(x.size): # Loop over all particles.
        if other == particle:
            continue # A particle does not interact with itself.

        # Measure the distance using the closest periodic copy.
        dx = minimum_image(x[particle] - x[other], box_length)
        dy = minimum_image(y[particle] - y[other], box_length)
        distance_sq = dx * dx + dy * dy

        # Squared distances avoid an unnecessary square root in every test.
        if distance_sq < sigma0_sq:
            return -1
        if distance_sq < sigma1_sq:
            energy += 1 # epsilon = 1 in reduced units
    return energy


@njit
def total_energy(x, y, box_length):
    """
    Calculate the complete system energy for initialization and checks.

    Only pairs with j > i are visited, so every interaction is counted once.
    """

    energy = 0
    sigma0_sq = SIGMA_0**2
    sigma1_sq = SIGMA_1**2

    for i in range(x.size - 1):
        for j in range(i + 1, x.size):
            dx = minimum_image(x[i] - x[j], box_length)
            dy = minimum_image(y[i] - y[j], box_length)
            distance_sq = dx * dx + dy * dy

            if distance_sq < sigma0_sq:
                return -1

            if distance_sq < sigma1_sq:
                energy += 1

    return energy


def random_nonoverlapping_positions(n_particles, box_length, rng):
    """
```

```
    Construct a random starting configuration without hard-core overlaps.

    Particles are inserted one after another. A candidate is discarded if it
    overlaps any particle that has already been placed.
    """

    # np.empty allocates the arrays without filling them with zeros.
    x = np.empty(n_particles)
    y = np.empty(n_particles)

    for i in range(n_particles):

        while i <= n_particles:
            # Draw two independent uniform coordinates.
            candidate_x, candidate_y = rng.uniform(0.0, box_length, 2)

            if i == 0:
                valid = True

            else:
                # x[:i] is a slice containing only particles placed so far.
                # Naive distance
                dx = candidate_x - x[:i]
                dy = candidate_y - y[:i]

                # Apply the minimum-image convention (periodic boundary conditions) to all
                    distances at once.
                dx -= box_length * np.rint(dx / box_length)
                dy -= box_length * np.rint(dy / box_length)

                # np.all is True only if every pair satisfies the condition.
                valid = np.all(dx * dx + dy * dy >= SIGMA_0**2)

            if valid:
                x[i], y[i] = candidate_x, candidate_y
                break # Stop searching after a valid position was found.

    return x, y


@njit
def accumulate_pair_histogram(x, y, box_length, histogram):
    """
    Applied for a chosen configuration: unordered pair distances are added to the RDF histogram.

    Distances larger than L/2 are excluded because circular shells are no
    longer represented without geometric complications in a square box.

    """

    maximum_r = box_length / 2.0
    bin_width = maximum_r / histogram.size

    for i in range(x.size - 1):
        for j in range(i + 1, x.size):

            dx = minimum_image(x[i] - x[j], box_length)
            dy = minimum_image(y[i] - y[j], box_length)
            r = np.sqrt(dx * dx + dy * dy)

            if r < maximum_r:
                # int truncates a positive number, selecting its histogram bin.
                bin_index = int(r / bin_width)
```

```
                histogram[bin_index] += 1


@njit
def run_stage(x, y, box_length, temperature, maximum_displacement,
              n_steps, energy_every, rdf_every, collect_rdf, seed):
    """
    Run one equilibration, tuning, or production stage.

    The x and y arrays are modified in place, so the returned final state is
    already stored in the input arrays. RDF sampling is optional because it
    is needed only during production.
    """

    # Configs and allocations
    np.random.seed(seed)
    n_energy_samples = n_steps // energy_every

    energy_steps = np.empty(n_energy_samples, dtype=np.int64)
    energies = np.empty(n_energy_samples, dtype=np.int64)
    histogram = np.zeros(RDF_BINS, dtype=np.int64)

    accepted = 0
    unfavorable_accepted = 0
    hard_rejected = 0
    metropolis_rejected = 0
    energy_sample = 0
    rdf_frames = 0

    # Exact starting energy
    energy = total_energy(x, y, box_length)

    for step in range(1, n_steps + 1):

        # randint chooses one particle index uniformly from 0 to N-1.
        particle = np.random.randint(x.size)
        old_x, old_y = x[particle], y[particle]
        old_particle_energy = particle_energy(x, y, particle, box_length)

        # Propose displacement. The modulo wraps coordinates back into the box.
        x[particle] = (old_x + maximum_displacement*np.random.uniform(-1, 1)) % box_length
        y[particle] = (old_y + maximum_displacement*np.random.uniform(-1, 1)) % box_length

        new_particle_energy = particle_energy(x, y, particle, box_length)

        if new_particle_energy < 0:
            # Hard-core overlaps are forbidden and therefore always rejected.
            x[particle], y[particle] = old_x, old_y
            hard_rejected += 1

        else:
            delta_energy = new_particle_energy - old_particle_energy

            # Favorable/neutral moves are accepted. Unfavorable moves are
            # accepted with the Metropolis probability exp(-Delta U / T*).
            if delta_energy <= 0 or np.random.random() < np.exp(-delta_energy / temperature):
                energy += delta_energy
                accepted += 1
                if delta_energy > 0:
                    unfavorable_accepted += 1

            else:
                # rejected proposal
```

```
                x[particle], y[particle] = old_x, old_y
                metropolis_rejected += 1

        # Energy at regular intervals
        if step % energy_every == 0:
            energy_steps[energy_sample] = step
            energies[energy_sample] = energy
            energy_sample += 1

        # Add the current complete configuration to g(r) at regular intervals
        if collect_rdf and step % rdf_every == 0:
            accumulate_pair_histogram(x, y, box_length, histogram)
            rdf_frames += 1

    return (energy_steps, energies, histogram, rdf_frames, accepted,
            unfavorable_accepted, hard_rejected, metropolis_rejected, energy)


def tune_displacement(x, y, box_length, temperature, initial_displacement, seed, tuning_steps=10
    _000):
    """
    Tune the proposal size until the acceptance rate lies in the target range.

    Its initial burn-in and all tuning blocks are discarded from measurements.
    The final displacement remains fixed throughout production.
    """

    displacement = initial_displacement
    print(f" tuning burn-in: {TUNING_BURN_IN_STEPS} discarded moves")

    # First the system evolves without evaluating.
    run_stage(
        x, y, box_length, temperature, displacement,
        TUNING_BURN_IN_STEPS, TUNING_BURN_IN_STEPS,
        TUNING_BURN_IN_STEPS, False, seed)

    for tuning_round in range(MAX_TUNING_ROUNDS):

        test = run_stage(
            x, y, box_length, temperature, displacement,
            tuning_steps, tuning_steps, tuning_steps, False,
            seed + tuning_round + 1)

        # test[4] is the number of accepted moves returned by run_stage
        acceptance_rate = test[4] / tuning_steps
        print(f" tuning {tuning_round + 1}: displacement={displacement:.4f}, acceptance={
            acceptance_rate:.2%}")

        in_target_range = MIN_ACCEPTANCE <= acceptance_rate <= MAX_ACCEPTANCE

        # several blocks so that one noisy block cannot end tuning early
        if in_target_range and tuning_round + 1 >= MIN_TUNING_ROUNDS:
            return displacement

        if tuning_round == MAX_TUNING_ROUNDS - 1:
            print(" warning: target acceptance range was not reached")
            return displacement

        # Smaller displacements increase acceptance, larger ones lower it.
        if acceptance_rate < MIN_ACCEPTANCE:
            displacement *= 0.8
        elif acceptance_rate > MAX_ACCEPTANCE:
            displacement *= 1.2
```

```
303
304     return displacement
305
306
307 def normalize_rdf(histogram, frames, n_particles, box_length):
308     """
309     Convert all accumulated pair counts into the radial distribution g(r).
310
311     Division by
312     area, (-> local number density in the shell)
313     number density, (-> compares local density to average density)
314     n_particles, (-> average over all particles)
315     and frame count (-> average over all frames)
316     makes g(r) dimensionless.
317     The factor 2 converts unordered pairs (counted once) into
318     neighbour relations (each pair contributes to both particles (count for ij contributes to
            particle i AND j)).
319     """
320
321     edges = np.linspace(0.0, box_length / 2.0, histogram.size + 1)
322
323     # Each r value is placed at the center of its bin
324     centers = 0.5 * (edges[:-1] + edges[1:])
325     areas = np.pi * (edges[1:]**2 - edges[:-1]**2)
326     number_density = n_particles / box_length**2
327     g_r = 2.0 * histogram / (frames * n_particles * number_density * areas)
328     return centers, g_r
329
330
331
332 ###############################################################################
333
334 # ----------------------------- Main Run ---------------------------------
335
336 ###############################################################################
337
338 def simulate_density(density, case_number):
339     """Run the complete workflow for one reduced density."""
340     # From rho* = N sigma_0^2/L^2 and sigma_0 = 1, L = sqrt(N/rho*).
341     box_length = np.sqrt(N / density)
342     output_directory = OUTPUT_DIR / f"rho_{density:.3f}"
343     # parents=True creates missing parent folders; exist_ok avoids an error on reruns.
344     output_directory.mkdir(parents=True, exist_ok=True)
345
346     # default_rng is the random-number generator.
347     rng = np.random.default_rng(BASE_SEED + case_number)
348     x, y = random_nonoverlapping_positions(N, box_length, rng)
349
350     print(f"\nDensity {density:.3f}, L={box_length:.3f}")
351     start_time = time.time()
352     maximum_displacement = MAX_DISPLACEMENT
353
354     # Equilibration is recorded for the diagnostic energy plot, but not for g(r).
355     equilibration = run_stage(
356         x, y, box_length, TEMPERATURE, maximum_displacement,
357         EQUILIBRATION_STEPS, ENERGY_EVERY, RDF_EVERY, False,
358         BASE_SEED + 1 + case_number)
359
360     eq_steps, eq_energies = equilibration[0], equilibration[1]
361
362     print("Displacement tuning:")
363     maximum_displacement = tune_displacement(
364         x, y, box_length, TEMPERATURE, maximum_displacement,
```

```
        BASE_SEED + case_number, TUNING_STEPS)

    # Production uses the fixed tuned displacement and accumulates the RDF.
    production = run_stage(
        x, y, box_length, TEMPERATURE, maximum_displacement,
        PRODUCTION_STEPS, ENERGY_EVERY, RDF_EVERY, True,
        BASE_SEED + 2 + case_number)

    (_, prod_energies, histogram, frames, accepted, unfavorable_accepted,
     hard_rejected, metropolis_rejected, tracked_energy) = production

    # recalculate U to detect errors in energy updates
    recalculated_energy = total_energy(x, y, box_length)
    energy_difference = abs(tracked_energy - recalculated_energy)

    acceptance_ratio = accepted / PRODUCTION_STEPS
    unfavorable_fraction = unfavorable_accepted / PRODUCTION_STEPS
    unfavorable_of_accepted = unfavorable_accepted / accepted

    r, g_r = normalize_rdf(histogram, frames, N, box_length)

    ######################################################################

    # Save all results to disk

    ######################################################################

    np.savetxt(output_directory / "equilibration_energy.csv",
               np.column_stack((eq_steps, eq_energies / N)), delimiter=",",
               header="attempted_moves,energy_per_particle", comments="")

    np.savetxt(output_directory / "rdf.csv", np.column_stack((r, g_r)),
               delimiter=",", header="r,g_r", comments="")

    np.savetxt(output_directory / "final_coordinates.csv",
               np.column_stack((x, y)), delimiter=",", header="x,y", comments="")

    summary = (
        f"N = {N}\nbox length = {box_length:.10g}\ndensity = {density}\n"
        f"T* = {TEMPERATURE}\nmaximum displacement = {maximum_displacement:.8g}\n"
        f"tuning burn-in attempts = {TUNING_BURN_IN_STEPS}\n"
        f"tuning attempts per round = {TUNING_STEPS}\n"
        f"equilibration attempts = {EQUILIBRATION_STEPS}\n"
        f"production attempts = {PRODUCTION_STEPS}\nrdf frames = {frames}\n"
        f"overall acceptance = {accepted}/{PRODUCTION_STEPS} ({acceptance_ratio:.6%})\n"
        f"unfavorable moves accepted = {unfavorable_accepted} "
        f"({unfavorable_fraction:.6%} of all attempts; "
        f"{unfavorable_of_accepted:.6%} of accepted moves)\n"
        f"hard-core rejected = {hard_rejected} ({hard_rejected / PRODUCTION_STEPS:.6%})\n"
        f"Metropolis rejected = {metropolis_rejected} "
        f"({metropolis_rejected / PRODUCTION_STEPS:.6%})\n"
        f"tracked final energy = {tracked_energy}\n"
        f"recalculated final energy = {recalculated_energy}\n"
        f"absolute energy discrepancy = {energy_difference}\n"
        f"mean production U/N = {np.mean(prod_energies / N):.8g}\n"
        f"runtime seconds = {time.time() - start_time:.2f}\n")

    (output_directory / "summary.txt").write_text(summary, encoding="utf-8")
    print(summary)

    return acceptance_ratio, energy_difference
```

```python
def main():
    """Run all densities sequentially."""
    OUTPUT_DIR.mkdir(exist_ok=True)

    rows = []
    for case_number, density in enumerate(DENSITIES):
        acceptance_ratio, energy_difference = simulate_density(density, case_number)
        rows.append([density, acceptance_ratio, energy_difference])

    rows = np.array(rows)
    np.savetxt(OUTPUT_DIR / "acceptance_and_energy_summary.csv", rows, delimiter=",",
               header="density,acceptance_ratio,energy_discrepancy", comments="")


# This guard runs main only when this file is executed directly. It does not
# run when mc_temperature_drop_parallel.py imports the functions from this module.
if __name__ == "__main__":
    main()
```

mc_temperature_drop_parallel.py

```python
"""
Parallel temperature-drop MC with 10 independent chains.

Runs 10 independent chains, all following the same temperature schedule
(only the random seed differs between chains).
Usage:
  python mc_temperature_drop_parallel.py

"""

from pathlib import Path
import time
import sys
from io import StringIO
from multiprocessing import Pool, Manager
from functools import partial
import numpy as np
import mc_fixed_temperature as mc


DENSITY = 0.291
MAX_DISPLACEMENT = 0.5
EQUILIBRATION_STEPS = 5_000_000
ENERGY_EVERY = 1_000
BASE_SEED = 2026
OUTPUT_DIR = Path("temperature_drop_results_parallel")

# Configuration snapshots (x, y) are saved at these temperatures
SNAPSHOT_TEMPERATURES = (0.25, 0.20, 0.19, 0.18, 0.15)

# Production steps per temperature window (T_high, T_low, steps)
PRODUCTION_STEPS_WINDOWS = [
    (0.250, 0.225, 50_000_000),
    (0.225, 0.200, 150_000_000),
    (0.200, 0.170, 300_000_000),
    (0.170, 0.150, 150_000_000)]


def base_temperature_schedule():
    """
    Returns numpy array of temperatures in descending order.

```

```
    Every chain uses this same schedule, only the seed differs between chains.
    """
    temps = []
    temps.extend(np.arange(250, 205, -10) / 1000)
    temps.extend(np.arange(205, 175, -2) / 1000)
    temps.extend(np.arange(175, 149, -10) / 1000)
    temps.append(0.150)
    return np.array(sorted(temps, reverse=True))


def production_steps_schedule(schedule):
    """Returns array of production steps for each temperature in the schedule."""
    steps = []
    for temperature in schedule:
        steps.append(next(n for t_high, t_low, n in PRODUCTION_STEPS_WINDOWS
                          if t_low <= temperature <= t_high))
    return np.array(steps)


def snapshot_indices(schedule, targets):
    """
    Returns the sorted schedule indices closest to each target temperature.
    """
    return sorted({int(np.argmin(np.abs(schedule - target))) for target in targets})


# CORE: Single-chain runner

def run_single_chain(chain_id, seed, temperatures, production_steps, box_length,
                     snapshot_steps, total_runs, progress_counter, progress_lock):
    """Runs a single chain of the temperature-drop simulation with a given seed."""

    rng = np.random.default_rng(seed)
    x, y = mc.random_nonoverlapping_positions(mc.N, box_length, rng)

    results = []
    maximum_displacement = MAX_DISPLACEMENT

    for index, temperature in enumerate(temperatures):
        n_production = int(production_steps[index])
        old_stdout = sys.stdout # save current stdout
        sys.stdout = StringIO() # suppress output for mc run (loads of print statements)

        # find max displacement for this temperature
        maximum_displacement = mc.tune_displacement(x, y, box_length, temperature,
            maximum_displacement, seed + 3 * index)

        # equilibration stage (evolves x, y in place)
        mc.run_stage(x, y, box_length, temperature, maximum_displacement,
            EQUILIBRATION_STEPS, ENERGY_EVERY, ENERGY_EVERY, False, seed + 3 * index + 1)

        # production stage
        production = mc.run_stage(
            x, y, box_length, temperature, maximum_displacement,
            n_production, ENERGY_EVERY, ENERGY_EVERY, False, seed + 3 * index + 2)

        sys.stdout = old_stdout # restore stdout

        # data unpacking
        production_energies = production[1].astype(float)
        accepted = production[4]
        unfavorable_accepted = production[5]
        tracked_energy = production[8]
```

```
105
106         # Variance and heat capacity calculations
107         mean_energy = np.mean(production_energies)
108         mean_energy_squared = np.mean(production_energies**2)
109         energy_fluctuation = mean_energy_squared - mean_energy**2
110         heat_capacity_over_k = energy_fluctuation / (temperature**2)
111         heat_capacity_per_particle = heat_capacity_over_k / mc.N
112         acceptance_rate = accepted / n_production
113
114         # total energy sanity check
115         recalculated_energy = mc.total_energy(x, y, box_length)
116         energy_discrepancy = abs(tracked_energy - recalculated_energy)
117
118         # Only needed temperatures get a saved snapshot.
119         if index in snapshot_steps:
120             snapshot_filename = OUTPUT_DIR / f"chain_{chain_id}" / f"prod_T_{temperature:.3f}.npz
                "
121             snapshot_filename.parent.mkdir(parents=True, exist_ok=True)
122             np.savez_compressed(snapshot_filename, x=x, y=y)
123
124         # store results for this temperature
125         results.append([
126             temperature, mean_energy, mean_energy / mc.N, energy_fluctuation,
127             heat_capacity_over_k, heat_capacity_per_particle,
128             acceptance_rate, unfavorable_accepted, energy_discrepancy,
129             maximum_displacement, n_production, accepted,
130             tracked_energy, recalculated_energy])
131
132         # Update progress counter and print progress
133         with progress_lock:
134             progress_counter.value += 1
135             print(f"{progress_counter.value}/{total_runs} MC runs done", flush=True)
136
137     return np.asarray(results)
138
139
140
141 ######################################################
142 # EXPORT + MAIN()
143 # everything is written to disk
144 ######################################################
145
146 RESULT_COLUMNS = [
147     "temperature", "mean_energy", "mean_energy_per_particle", "energy_fluctuation",
148     "heat_capacity_over_k", "heat_capacity_per_particle", "acceptance_rate",
149     "unfavorable_accepted", "energy_discrepancy", "max_displacement",
150     "production_steps", "accepted_moves", "tracked_energy", "recalculated_energy",
151 ]
152
153
154 def main():
155     OUTPUT_DIR.mkdir(exist_ok=True)
156
157     # 10 seeds
158     num_seeds = 10
159     schedule = base_temperature_schedule()
160     production_steps = production_steps_schedule(schedule)
161     snapshot_steps = snapshot_indices(schedule, SNAPSHOT_TEMPERATURES)
162     box_length = np.sqrt(mc.N / DENSITY)
163     seeds = [BASE_SEED + seed_idx * 1000 for seed_idx in range(num_seeds)]
164     total_runs = num_seeds * len(schedule)
165
```

```
    manager = Manager() # for shared progress counter otherwise multiprocessing.Pool has no shared state
    # partial function to fix arguments for run_single_chain
    run = partial(run_single_chain, total_runs=total_runs,
                  progress_counter=manager.Value('i', 0), progress_lock=manager.Lock())

    # Create a list of tasks that will be executed in parallel. Each task is a tuple of arguments for run_single_chain.
    tasks = [(f"seed_{i}", seeds[i], schedule, production_steps, box_length, snapshot_steps)
             for i in range(num_seeds)]

    # Run the tasks in parallel using a pool of processes. The results from all chains are collected in all_results.
    start = time.time()
    with Pool(num_seeds) as pool:
        all_results = pool.starmap(run, tasks)
    total_time = time.time() - start

    # one table for all chains
    table = np.vstack([np.column_stack((np.full(len(r), i), r)) for i, r in enumerate(all_results)])
    np.savetxt(OUTPUT_DIR / "all_chains_results.csv", table, delimiter=",", fmt="%.17g",
               header="chain," + ",".join(RESULT_COLUMNS), comments="")

    print(f"Runtime: {total_time/3600:.2f}h")

if __name__ == "__main__":
    main()
```

plot_results.py

```
"""
Plotting script for the stripe-phase MC project.

Reads the CSV/NPZ files written by mc_fixed_temperature.py and
mc_temperature_drop_parallel.py and creates every figure for the report.
"""

from pathlib import Path

import matplotlib.pyplot as plt
from matplotlib.collections import EllipseCollection
import numpy as np


plt.rcParams.update({
    "figure.dpi": 120,
    "savefig.dpi": 300,
    "font.size": 11,
    "axes.titlesize": 12,
    "axes.labelsize": 11,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "legend.fontsize": 9,
})

# Must match the MC scripts
N = 250
SIGMA_0, SIGMA_1 = 1.0, 2.5
GREEN = plt.cm.tab10(2)

FIXED_DENSITIES = (0.100, 0.150, 0.227, 0.291, 0.380)
FIXED_DIR = Path("fixed_temperature_results")
```

```
33
34 DROP_DENSITY = 0.291
35 DROP_DIR = Path("temperature_drop_results_parallel")
36 SNAPSHOT_TEMPERATURES = (0.25, 0.20, 0.19, 0.18, 0.15)
37
38
39 # --------------------- particles drawn to true size + aura -----------------
40 def with_periodic_images(x, y, box_length, margin):
41     """Add periodic copies near the box edges so wrapped particles are whole."""
42     xs, ys = [x], [y]
43     for shift_x in (-1, 0, 1):
44         for shift_y in (-1, 0, 1):
45             if shift_x == 0 and shift_y == 0:
46                 continue
47             xi, yi = x + shift_x * box_length, y + shift_y * box_length
48             keep = (xi > -margin) & (xi < box_length + margin) & (yi > -margin) & (yi <
                   box_length + margin)
49             xs.append(xi[keep])
50             ys.append(yi[keep])
51     return np.concatenate(xs), np.concatenate(ys)
52
53
54 def draw_particles(ax, x, y, box_length, color, title):
55     """
56     Draw every particle to scale: a solid core (diameter sigma_0) inside a
57     translucent aura (diameter sigma_1).
58     """
59     xs, ys = with_periodic_images(x, y, box_length, SIGMA_1)
60     offsets = np.column_stack((xs, ys))
61
62     aura = EllipseCollection(
63         np.full(xs.size, SIGMA_1), np.full(xs.size, SIGMA_1), 0,
64         units="xy", offsets=offsets, offset_transform=ax.transData,
65         facecolors=(*color[:3], 0.15), edgecolors=(*color[:3], 0.4), linewidths=0.5)
66     core = EllipseCollection(
67         np.full(xs.size, SIGMA_0), np.full(xs.size, SIGMA_0), 0,
68         units="xy", offsets=offsets, offset_transform=ax.transData,
69         facecolors=color, edgecolors="none")
70
71     # Clip to the simulation box so the periodic images outside don't show.
72     clip = plt.Rectangle((0, 0), box_length, box_length, transform=ax.transData)
73     clip.set_visible(False)
74     ax.add_patch(clip)
75     for collection in (aura, core):
76         ax.add_collection(collection)
77         collection.set_clip_path(clip)
78
79     ax.plot([0, box_length, box_length, 0, 0], [0, 0, box_length, box_length, 0],
80             color="0.4", linewidth=0.8)
81     ax.set_xlim(-0.02 * box_length, 1.02 * box_length)
82     ax.set_ylim(-0.02 * box_length, 1.02 * box_length)
83     ax.set_aspect("equal")
84     ax.set_xlabel(r"$x/\sigma_0$")
85     ax.set_ylabel(r"$y/\sigma_0$")
86     ax.set_title(title)
87
88
89 # ------------------------ fixed-temperature plots -------------------------
90 def plot_equilibration_by_density():
91     """Energy per particle vs. attempted moves, one panel per density."""
92     fig, axes = plt.subplots(1, len(FIXED_DENSITIES), figsize=(3.0 * len(FIXED_DENSITIES), 4.2),
93                              sharex=True, sharey=True)
94     for ax, density in zip(axes, FIXED_DENSITIES):
```

```
        steps, energy = np.loadtxt(FIXED_DIR / f"rho_{density:.3f}" / "equilibration_energy.csv",
                                   delimiter=",", skiprows=1, unpack=True)
        ax.plot(steps, energy, linewidth=1.8, color=GREEN)
        ax.set_title(rf"$\rho^*={density:.3f}$")
        ax.set_xlabel("attempted moves")
    axes[0].set_ylabel(r"$U/N$")
    fig.tight_layout()
    fig.savefig(FIXED_DIR / "equilibration_energy.png")
    plt.close(fig)


def plot_structures_and_rdf():
    """Equilibrium structures (row 1) and g(r) (row 2), one column per density."""
    fig, axes = plt.subplots(2, len(FIXED_DENSITIES), figsize=(3.2 * len(FIXED_DENSITIES), 7.0))

    for idx, density in enumerate(FIXED_DENSITIES):
        directory = FIXED_DIR / f"rho_{density:.3f}"
        box_length = np.sqrt(N / density)

        x, y = np.loadtxt(directory / "final_coordinates.csv", delimiter=",", skiprows=1, unpack=
            True)
        draw_particles(axes[0, idx], x, y, box_length, GREEN, rf"$\rho^*={density:.3f}$")

        r, g_r = np.loadtxt(directory / "rdf.csv", delimiter=",", skiprows=1, unpack=True)
        ax = axes[1, idx]
        ax.plot(r, g_r, color=GREEN, linewidth=1.8)
        ax.axhline(1.0, color="0.5", linestyle="--", linewidth=1)
        ax.axvline(SIGMA_0, color="0.3", linestyle=":", linewidth=1.8)
        ax.axvline(SIGMA_1, color="0.3", linestyle="-.", linewidth=1.8)
        ax.set_xlabel(r"$r/\sigma_0$")

    axes[1, 0].set_ylabel(r"$g(r)$")
    fig.tight_layout()
    fig.savefig(FIXED_DIR / "equilibrium_structures_and_rdf.png")
    plt.close(fig)


# ------------------------- temperature-drop plots ---------------------------
def load_chain_results():
    """Load all_chains_results.csv into {chain index: structured row array}."""
    table = np.genfromtxt(DROP_DIR / "all_chains_results.csv", delimiter=",", names=True)
    return {chain: table[table["chain"] == chain] for chain in np.unique(table["chain"]).astype(
        int)}


def plot_seed0_configurations(box_length):
    """Cooling sequence of realization 1 (seed_0), one panel per saved temperature."""
    chain_dir = DROP_DIR / "chain_seed_0"
    available = np.array(sorted(float(f.stem.split("_T_")[1]) for f in chain_dir.glob("prod_T_*.
        npz")))

    fig, axes = plt.subplots(1, len(SNAPSHOT_TEMPERATURES),
                             figsize=(3.2 * len(SNAPSHOT_TEMPERATURES), 3.6))

    for ax, target in zip(axes, SNAPSHOT_TEMPERATURES):
        temperature = available[np.argmin(np.abs(available - target))]
        with np.load(chain_dir / f"prod_T_{temperature:.3f}.npz") as d:
            x, y = d["x"], d["y"]
        draw_particles(ax, x, y, box_length, GREEN, rf"$T^*={temperature:.2f}$")

    fig.tight_layout()
    fig.savefig(DROP_DIR / "configurations_realisation1.png")
    plt.close(fig)
```

```python


def draw_chain_comparison(ax, data, column, ylabel):
    """
    Every realization as a thin dashed line, plus mean +- standard error
    across all realizations as a thicker black line.
    """
    chains = sorted(data.keys())

    for idx, chain in enumerate(chains):
        rows = data[chain]
        ax.plot(rows["temperature"], rows[column], linestyle="--", marker="o",
                markersize=3, linewidth=0.6, alpha=0.6, color=plt.cm.tab10(idx),
                label=f"Realisation {idx + 1}")

    stacked = np.array([data[chain][column] for chain in chains])
    temperatures = data[chains[0]]["temperature"]
    mean = stacked.mean(axis=0)
    standard_error = stacked.std(axis=0, ddof=1) / np.sqrt(len(chains))
    ax.errorbar(temperatures, mean, yerr=standard_error, fmt="o-", color="black",
                markersize=6, linewidth=1.6, capsize=3, zorder=5, label="Mean ±SEM")

    ax.set(xlabel=r"Temperature $T^*$", ylabel=ylabel)


def plot_energy_and_cv(data):
    """Heat capacity and energy per particle side by side, both vs. temperature."""
    fig, (ax_cv, ax_u) = plt.subplots(1, 2, figsize=(12, 4.5),
                                      gridspec_kw={"width_ratios": [2, 1]})

    draw_chain_comparison(ax_cv, data, "heat_capacity_per_particle", r"Heat capacity $C_V/(Nk)$"
        )
    draw_chain_comparison(ax_u, data, "mean_energy_per_particle", r"Energy per particle $U/N$")
    ymin, ymax = ax_u.get_ylim()
    ax_u.set_yticks(np.arange(np.ceil(ymin * 10) / 10, ymax, 0.1))
    ax_cv.legend(loc="upper left", ncol=2, fontsize=7)

    fig.tight_layout()
    fig.savefig(DROP_DIR / "energy_and_cv_vs_temperature.png")
    plt.close(fig)

plot_equilibration_by_density()
plot_structures_and_rdf()
plot_seed0_configurations(np.sqrt(N / DROP_DENSITY))
plot_energy_and_cv(load_chain_results())
print(f"All plots done.")
```